ASSIGNMENT - 2.5

Name: P. Sanjay Kumar
Roll No: 2403A52L06
Batch-50

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju		
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week1 – Wednesday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches
Assignment Number:1.3(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*Expected Time to complete*

1	Lab 2: Exploring Additional AI Coding Tools beyond Copilot – Gemini (Colab) and Cursor AI **Lab Objectives:** ❖ To explore and evaluate the functionality of Google Gemini for AI-assisted coding within Google Colab. ❖ To understand and use Cursor AI for code generation, explanation, and refactoring. ❖ To compare outputs and usability between Gemini, GitHub Copilot, and Cursor AI. ❖ To perform code optimization and documentation using AI tools. **Lab Outcomes (LOs):** After completing this lab, students will be able to: ❖ Generate Python code using Google Gemini in Google Colab. ❖ Analyze the effectiveness of code explanations and suggestions by Gemini. ❖ Set up and use Cursor AI for AI-powered coding assistance. ❖ Evaluate and refactor code using Cursor AI features. ❖ Compare AI tool behavior and code quality across different platforms.	Week1 - Monday
	Task 1: Refactoring Odd/Even Logic (List Version) ❖ **Scenario:** You are improving legacy code. ❖ **Task:** Write a program to calculate the sum of odd and even numbers in a list, then refactor it using AI. ❖ **Expected Output:** ❖ Original and improved code **Prompt:** Hey, I have this old Python script that uses a basic for-loop and range(len()) to find the sum of odd and even numbers in a list. It feels a bit clunky. Can you refactor this to be more 'Pythonic' using list comprehensions or built-in functions? Show me the original vs the new version. Code:	

lab assignment2.py > ...

```python
def calculate_sums(numbers):
    even_sum = 0
    odd_sum = 0

    for num in numbers:
        if num % 2 == 0:
            even_sum += num
        else:
            odd_sum += num

    return even_sum, odd_sum
numbers = list(map(int, input("Enter numbers separated by space: ").split()))
even, odd = calculate_sums(numbers)
print("Even Sum:", even)
print("Odd Sum:", odd)
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\sanjay\AI Assistant coding> & C:\Users\
oding/lab assignment2.py"
Enter numbers separated by space: 1 2 3 4 5 6 7 8
Even Sum: 20
Odd Sum: 16
PS C:\Users\sanjay\AI Assistant coding>
```

JUSTIFICATION:

This works because it gives the AI a role (cleaner of legacy code) and specifies a target style (Pythonic/List Comprehension). This prevents the AI from just giving you the same code back and forces it to optimize for readability.

Task 2: Area Calculation Explanation

- **Scenario:**
  You are onboarding a junior developer.

- **Task:**
  Ask Gemini to explain a function that calculates the area of different shapes.

- **Expected Output:**
  - Code
  - Explanation

Prompt:

"I'm onboarding a junior developer today. Can you write a Python function that calculates the area for a circle, rectangle, and triangle? After the code,

give a simple, friendly explanation of how it works—especially why we use math.pi and how we handle different inputs—so the junior can follow along easily.

Code:

```
import math
def calculate_area(shape, d1, d2=None):
    shape = shape.lower()
    if shape == "circle":
        return math.pi * d1 ** 2
    elif shape == "rectangle":
        return d1 * d2
    elif shape == "triangle":
        return 0.5 * d1 * d2
    else:
        return None
# ◆ Take input from the user
shape = input("Enter shape (circle, rectangle, triangle): ").lower()

if shape == "circle":
    radius = float(input("Enter radius: "))
    area = calculate_area(shape, radius)
elif shape == "rectangle":
    length = float(input("Enter length: "))
    breadth = float(input("Enter breadth: "))
    area = calculate_area(shape, length, breadth)
elif shape == "triangle":
    base = float(input("Enter base: "))
    height = float(input("Enter height: "))
    area = calculate_area(shape, base, height)
else:
    area = None

if area is not None:
    print(f"The area of the {shape} is: {area}")
else:
    print("Invalid shape entered")
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\sanjay\AI Assistant coding> & C:\Users
oding/lab assignment2.py"
Enter shape (circle, rectangle, triangle): circle
Enter radius: 3
The area of the circle is: 28.274333882308138
PS C:\Users\sanjay\AI Assistant coding>
```

JUSTIFICATION:
By setting a persona (onboarding a junior), you ensure the AI doesn't just give a technical summary. It will use simpler language and focus on the "why" behind the code, which is exactly what your lab task requires**.**

Task 3: Prompt Sensitivity Experiment

- **Scenario:**
  You are testing how AI responds to different prompts.

- **Task:**
  Use Cursor AI with different prompts for the same problem and observe code changes.

- **Expected Output:**
  - **Prompt list**
  - **Code variations**

Prompt:
A: "Write a function to find the max value in a list." Prompt B: "Write a function to find the max value in a list, but do it manually using a for-loop and don't use any built-in functions like max().

Code:

```python
def find_max(numbers):
    return max(numbers)
nums = [10, 25, 7, 40, 18]
print(find_max(nums))
```

Output:
40

JUSTIFICATION:
This is a constraint-based experiment. Prompt A lets the AI be "lazy" and efficient, while Prompt B forces it to show its algorithmic logic. Comparing these shows you how much "control" you have over the AI's output by just changing a few words**.**

	Task 4: Tool Comparison Reflection ❖ **Scenario:** You must recommend an AI coding tool. ❖ **Task:** Based on your work in this topic, compare Gemini, Copilot, and Cursor AI for usability and code quality. ❖ **Expected Output:** Short written reflection Prompt: "I've been using Gemini in Colab, GitHub Copilot, and Cursor AI for these lab tasks. Can you help me summarize the differences? Focus on which one felt best for explaining code to a human and which one was better at refactoring my files. Give me a short paragraph I can use in my lab report. **Justification:** This is a synthesis prompt. It asks the AI to look back at the work you've done and help you verbalize your thoughts into a professional reflection. It's perfect for the "Conclusion" or "Reflection" section of your SRU lab report. **Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.**	